SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

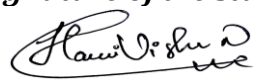
Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

*I **HARI VISHNU N / 192524319** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

INDUSTRY PROBLEM- SOLVING TASK

SET 01:
QUESTION 01:

Banking Transaction Processing System

A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

(i) Design suitable intermediate code for the given assignment and Boolean expressions. *(K3)*

(ii) Explain how backpatching can help in handling conditional transaction validation. *(K4)*

(iii) Analyze the role of intermediate languages in improving banking software portability. *(K5)*

SOLUTION:

BANKING TRANSACTION PROCESSING SYSTEM

[i] Design suitable intermediate code for assignment and Boolean expressions:

A compiler can use Three Address Code (TAC) as an intermediate representation. Each instruction contains at most three operands.

Consider the following statements:

```
balance = amount + interest
valid = (balance >= 1000) && (status == 1)
```

Three Address Code:

```
t1 = amount + interest
balance = t1

t2 = balance >= 1000
t3 = status == 1
t4 = t2 AND t3
valid = t4
```

For a transaction classification:

```
if (type == 1)
   deposit();
else if (type == 2)
   withdrawal();
else
   loan();
```

The intermediate code can be:

```
if type == 1 goto L1
goto L2

L1: call deposit
goto L4

L2: if type == 2 goto L3
goto L5

L3: call withdrawal
goto L4

L5: call loan

L4:
```

This representation makes the program easier for later optimization and target-code generation.

[ii] Backpatching for conditional transaction validation:

Backpatching is a compiler technique used to fill in the target addresses of jumps when the target location is not yet known during intermediate-code generation.

For example:

```
if (balance >= minimum && accountActive)
   approve();
else
   reject();
```

Initially, the compiler may generate:

```
100: if balance >= minimum goto _
101: goto _
102: if accountActive goto _
```

103: goto _
104: approve
105: goto _
106: reject

The _ represents an unknown target address.
After the complete Boolean expression and statement structure are generated,

backpatching fills these addresses:
100: if balance >= minimum goto 102
101: goto 106
102: if accountActive goto 104
103: goto 106
104: approve
105: goto 107
106: reject
107:

Thus, backpatching is useful for conditional transaction validation, because the compiler can generate conditional jumps first and determine their exact destinations later. It is particularly useful for Boolean expressions involving AND, OR, if-else, while, and case statements.

[iii] Role of intermediate languages in improving banking software portability:

An Intermediate Language (IL) acts as a machine-independent representation between the source program and the target machine.

The compilation process can be represented as:

Banking DSL Source Code
↓
Front End
↓
Intermediate Code
↓
Target-Specific Code
↓
Windows / Linux / Cloud / Mobile

The major benefits are:

1. Platform independence: The same intermediate code can be translated to different hardware architectures.
2. Reduced development effort: A compiler does not need a completely new front end for every target platform.
3. Optimization: Intermediate code allows optimizations such as constant folding, dead-code elimination and common-subexpression elimination.
4. Improved reliability: Optimized and standardized intermediate representations reduce errors during target-code generation.
5. Easy maintenance: Changes in the banking DSL can be handled mainly in the front end without rewriting every target-specific compiler component.
6. Cloud and legacy-system integration: Banking applications can be deployed across different servers and architectures using suitable target-code generators.
7. Efficient execution: Intermediate representations allow the compiler to optimize transaction-processing operations before generating machine code.

Conclusion:

For a banking transaction compiler, Three Address Code provides a suitable intermediate representation for assignments and Boolean expressions. Backpatching efficiently handles unresolved conditional jumps during transaction validation. An intermediate language separates source-language processing from machine-specific code generation, thereby improving portability, optimization, maintainability, and reliability of banking software.

SET 02
QUESTION 02:

Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development team intends to enhance compiler efficiency for faster and more reliable security analysis.

Tasks:

(i) Generate intermediate representation for network threat detection logic. *(K3)*

(ii) Discuss how case statements can efficiently classify multiple cyberattack scenarios. *(K4)*

(iii) Evaluate the importance of compiler optimization in real-time cybersecurity applications. *(K5)*

SOLUTION:

CYBERSECURITY INTRUSION DETECTION SYSTEM

[i] Generate intermediate representation for network threat detection logic.

For an intrusion detection system, Three Address Code (TAC) can be used as an intermediate representation.

Consider the threat detection logic:

```
if (packet_rate > 1000 && failed_login > 10)
   threat = 1;
else
   threat = 0;
```

The corresponding intermediate code is:

```
t1 = packet_rate > 1000
t2 = failed_login > 10
t3 = t1 AND t2

if t3 goto L1
goto L2

L1:
threat = 1
goto L3
```

```
L2:
threat = 0

L3:
```
Here, t1, t2, and t3 are temporary variables. This representation simplifies optimization and target-code generation.

Another example for detecting suspicious traffic:

```
if (source_unknown || port_scan)
   alert = 1;
TAC:
t1 = source_unknown
t2 = port_scan
t3 = t1 OR t2

if t3 goto L1
goto L2

L1: alert = 1
L2:
```

Thus, intermediate code converts complex Boolean threat-detection conditions into simple operations that can be efficiently processed.

[ii] Discuss how case statements can efficiently classify multiple cyberattack scenarios.

A case statement is useful when the intrusion detection system must classify a detected attack into several predefined categories.
For example:

```
switch (attack_type)
{
   case 1: alert = "DDoS";
         break;

   case 2: alert = "Brute Force";
         break;

   case 3: alert = "Port Scanning";
         break;

   case 4: alert = "SQL Injection";
```

```
        break;

    default: alert = "Unknown Attack";
}
```

The intermediate representation can be:

```
if attack_type == 1 goto L1
if attack_type == 2 goto L2
if attack_type == 3 goto L3
if attack_type == 4 goto L4
goto L5

L1: alert = "DDoS"
    goto L6

L2: alert = "Brute Force"
    goto L6

L3: alert = "Port Scanning"
    goto L6

L4: alert = "SQL Injection"
    goto L6

L5: alert = "Unknown Attack"

L6:
```

For a large number of attack types, the compiler can implement the case statement using a jump table. Instead of checking every case sequentially, the attack code is used to directly locate the corresponding target address.

Therefore, case statements provide:

- Faster classification of known attacks.
- Clear separation of attack categories.
- Efficient implementation using jump tables.
- Reduced number of conditional comparisons.
- Better scalability when many attack types are present.

This is particularly important for an IDS because thousands of network packets may need to be classified every second.

[iii] Evaluate the importance of compiler optimization in real-time cybersecurity applications.

Compiler optimization is extremely important in real-time intrusion detection because threat analysis must be performed with minimum latency.
Important optimizations include:

1. Constant Folding: Compile-time expressions are evaluated in advance, reducing runtime computation.
2. Dead Code Elimination: Unnecessary instructions are removed, reducing execution time.
3. Common Subexpression Elimination: Repeated Boolean or arithmetic expressions are calculated only once.
4. Control-Flow Optimization: Unnecessary jumps and branches are removed, making attack classification faster.
5. Code Generation Optimization: Efficient machine instructions are selected for rapid packet processing.
6. Register Allocation: Frequently accessed temporary values can be kept in CPU registers, reducing memory access.
7. Jump-table Optimization: Large case statements can be converted into jump tables for faster attack classification.

Overall Evaluation:
Without optimization, inefficient intermediate code can increase CPU usage, processing latency, and threat-detection delay. In cybersecurity, such delays can allow malicious traffic to pass through before an alert is generated.
Therefore, an optimized compiler improves:
Faster detection → Lower latency → Better resource utilization → Higher packet-processing rate → More reliable real-time security

Conclusion:
The IDS can use Three Address Code to represent Boolean and assignment operations, while case statements and jump tables efficiently classify different cyberattacks. Compiler optimizations are essential for reducing execution time and ensuring that threats are detected quickly and reliably in real-time network environments.

SET 03:
QUESTION 03:

Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically controls electricity distribution based on power consumption. The compiler in the embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:

(i) Generate intermediate code for load balancing and fault detection logic.
(ii) Explain the significance of backpatching in smart energy management systems.
(iii) Analyze how compiler optimization improves the performance of smart grids.

SOLUTION:

INTELLIGENT ENERGY MANAGEMENT SYSTEM:

[i] Generate intermediate code for load balancing and fault detection logic:

The compiler can use Three Address Code (TAC) as an intermediate representation.

Consider the load balancing and fault detection logic:

```
if (load > maximum_load && backup_available)
   switch_to_backup = 1;
else
   switch_to_backup = 0;

if (voltage < minimum_voltage || fault_detected)
   emergency_shutdown = 1;
```

The corresponding intermediate code is:

```
t1 = load > maximum_load
t2 = backup_available == 1
t3 = t1 AND t2

if t3 goto L1
goto L2

L1:
switch_to_backup = 1
```

```
goto L3

L2:
switch_to_backup = 0

L3:
t4 = voltage < minimum_voltage
t5 = fault_detected == 1
t6 = t4 OR t5

if t6 goto L4
goto L5

L4:
emergency_shutdown = 1
goto L6

L5:
emergency_shutdown = 0

L6:
```

Here, t1–t6 are temporary variables and L1–L6 are labels. This intermediate representation makes the logic easier to optimize and convert into machine instructions for the embedded controller.

[ii] Explain the significance of backpatching in smart energy management systems:

Backpatching is a compiler technique used to fill in the target addresses of jump instructions when their destinations are not yet known.
For example:

```
if (load > limit)
   reduce_power();
else
   increase_power();
```

During intermediate-code generation, the compiler may initially produce:

```
100: if load > limit goto _
101: goto _

102: reduce_power
103: goto _

104: increase_power
```

After the target labels are determined, backpatching fills the missing

addresses:
100: if load > limit goto 102
101: goto 104
102: reduce_power
103: goto 105
104: increase_power
105:

Significance in smart grids

1. Fast fault handling: Correct jump targets allow fault conditions to be processed immediately.
2. Efficient load redistribution: The controller can quickly select the appropriate power-management operation.
3. Supports complex conditions: Useful for if-else, Boolean expressions, loops, and switching operations.
4. Reduces code-generation complexity: Jump addresses can be resolved after the control-flow structure is known.
5. Improves reliability: Correct control flow prevents incorrect power-distribution decisions.

Therefore, efficient backpatching is important when the smart grid must respond rapidly to overloads, voltage fluctuations, or equipment failures.

[iii] Analyze how compiler optimization improves the performance of smart grids:

Compiler optimization transforms intermediate code into faster and more efficient target code. This is particularly important in embedded smart-grid controllers where processing power and response time may be limited.

Important optimizations:

1. Constant Folding: Calculates constant expressions during compilation instead of runtime.
2. Dead Code Elimination: Removes unused instructions, reducing program size and execution time.
3. Common Subexpression Elimination: Avoids calculating the same load or voltage condition repeatedly.
4. Control-Flow Optimization: Removes unnecessary jumps and improves the execution of fault-handling logic.

5. Register Allocation: Stores frequently used values such as voltage and load levels in CPU registers, reducing memory access.
6. Jump-Table Optimization: Makes multiple power-management choices faster when implemented using case statements.
7. Loop Optimization: Improves repeated monitoring operations used for continuous grid supervision.

Impact on smart-grid operation:
Efficient compiler optimization results in:
Optimized code → Faster processing → Faster fault detection → Quicker power redistribution → Lower energy wastage → Improved grid reliability
During peak-demand periods, these improvements can reduce the time required to detect overloads and activate backup or load-shedding mechanisms.

Conclusion:
Three Address Code provides a suitable intermediate representation for load balancing and fault detection. Backpatching ensures that conditional control flow is correctly resolved, enabling rapid responses to grid faults. Compiler optimization further reduces execution time and resource usage, making smart-grid controllers faster, more reliable, and energy-efficient.

SET 04:
QUESTION 04:

Automated Library Management System

A university is implementing an automated library management system to handle book issuance, returns, and digital catalog management. The compiler processes declarations, assignment statements, and procedure calls for member verification and book tracking operations. During semester examinations, inefficient intermediate code generation delayed transaction updates and fine calculations. The university plans to improve compiler efficiency for faster and more reliable library services.

Tasks:

(i) Develop intermediate code for book issue and return operations.
(ii) Explain how procedure calls improve modularity in library management systems.
(iii) Evaluate the significance of compiler optimization in educational information systems.

SOLUTION:

AUTOMATED LIBRARY MANAGEMENT SYSTEM

[i] Develop intermediate code for book issue and return operations

The compiler can use Three Address Code (TAC) as an intermediate representation.

Consider the following logic:

```
if (member_valid && book_available)
   issue_book();
else
   reject_issue();

if (book_returned)
   calculate_fine();
```

Intermediate code for book issue

```
t1 = member_valid == 1
t2 = book_available == 1
t3 = t1 AND t2

if t3 goto L1
goto L2

L1:
```

call issue_book
goto L3

L2:
call reject_issue

L3:
Intermediate code for book return and fine calculation
Assume:
fine = overdue_days * fine_per_day
The intermediate code is:
t4 = book_returned == 1
if t4 goto L4
goto L7

L4:
t5 = overdue_days * fine_per_day
fine = t5
call update_return
goto L7

L7:
Here, t1–t5 are temporary variables and L1–L7 are labels. The TAC simplifies the original library operations and makes them easier to optimize and convert into machine code.

[ii] Explain how procedure calls improve modularity in library management systems:

A procedure call allows a large library management system to be divided into smaller, independent modules.
For example:
call verify_member
call check_book
call issue_book
call calculate_fine
call update_catalog
Each procedure performs a specific task.

Advantages of procedure calls

1. Modularity: Each operation such as issuing, returning, and fine calculation can be implemented separately.

2. Code Reusability: The same procedure can be called whenever the operation is required.
3. Easy Maintenance: Changes to fine calculation do not require modifying the entire library system.
4. Reduced Complexity: Large library operations are divided into smaller manageable functions.
5. Error Isolation: Errors in one procedure can be identified and corrected without affecting unrelated modules.
6. Better Testing: Individual procedures can be tested independently.

For example, the same calculate_fine() procedure can be used for both student and faculty accounts with appropriate parameters.
Thus, procedure calls provide a structured and modular architecture for the library management system.

[iii] Evaluate the significance of compiler optimization in educational information systems:

Compiler optimization is important because educational systems may process a large number of transactions simultaneously, particularly during semester examinations, admissions, and result-processing periods.
Important optimizations include:

1. Constant Folding: Pre-computes fixed values such as standard fine rates.
2. Common Subexpression Elimination: Avoids repeatedly calculating the same values, such as overdue days.
3. Dead Code Elimination: Removes unnecessary instructions and reduces program size.
4. Control-Flow Optimization: Reduces unnecessary jumps in book issue and return operations.
5. Register Allocation: Keeps frequently used values in CPU registers for faster access.
6. Procedure Call Optimization: Reduces unnecessary overhead when frequently used library functions are called.
7. Code Size Optimization: Produces compact code, which is useful for embedded or resource-constrained educational systems.

Impact on the library system:
The overall effect can be represented as:
Compiler Optimization → Faster Execution → Faster Transactions → Quick Fine Calculation → Timely Catalog Updates → Better User Experience
For example, during examination periods, thousands of book issue/return requests may occur within a short time. Optimized intermediate and target code reduces processing delays and helps maintain accurate transaction records.

Conclusion:

Intermediate code such as Three Address Code provides an efficient representation for book issue, return, and fine calculations. Procedure calls divide the library system into reusable and maintainable modules. Compiler optimization reduces execution time and resource consumption, thereby providing faster, reliable, and scalable educational information services.

SET 05:
QUESTION 05:

AI-Based Customer Support Chatbot

A software company is building an AI-based customer support chatbot to handle user queries and provide automated responses. The compiler processes declarations, assignment statements, Boolean expressions, and procedure calls to analyze user requests and generate responses dynamically. During large-scale customer interactions, inefficient intermediate code generation caused delays in response delivery and reduced system performance. The organization plans to redesign the compiler to improve scalability and user experience.

Tasks:

(i) Design intermediate code for chatbot query processing and response generation.

(ii) Explain how procedure calls improve modularity in AI-based chatbot systems.

(iii) Analyze the importance of compiler optimization in intelligent customer support applications.

SOLUTION:

AI-BASED CUSTOMER SUPPORT CHATBOT

(i) Design intermediate code for chatbot query processing and response generation:

The compiler can use Three Address Code (TAC) to represent chatbot query-processing logic efficiently.

Consider the following logic:

```
if (user_query_valid && intent_detected)
   generate_response();
else
   request_clarification();
```

Intermediate Code

```
t1 = user_query_valid == 1
t2 = intent_detected == 1
t3 = t1 AND t2

if t3 goto L1
goto L2
L1:
call generate_response
goto L3
L2:
```

call request_clarification
L3:
For response generation:
response = template + user_data
send(response)
The intermediate code is:
t4 = template + user_data
response = t4
call send_response
Here, t1–t4 are temporary variables and L1–L3 are labels. This representation allows the compiler to optimize chatbot processing before generating machine code.

(ii) Explain how procedure calls improve modularity in AI-based chatbot systems:

Procedure calls divide the chatbot into independent modules, with each procedure performing a specific task.
For example:
call preprocess_query
call detect_intent
call analyze_sentiment
call generate_response
call send_response
Advantages

1. Modularity: Each chatbot function is implemented as a separate procedure.
2. Code Reusability: Procedures such as detect_intent() can be reused for different types of queries.
3. Easy Maintenance: Individual modules can be modified without changing the complete chatbot.
4. Simplified Testing: Each procedure can be tested independently.
5. Error Isolation: Problems in response generation can be debugged separately from query preprocessing.
6. Scalability: New capabilities such as language translation, sentiment analysis, or voice processing can be added as new modules.

Thus, procedure calls provide a structured, reusable, and maintainable architecture for AI-based chatbot systems.

(iii) Analyze the importance of compiler optimization in intelligent customer support applications:
Compiler optimization is essential when a chatbot handles thousands or millions of customer interactions simultaneously. Efficient code reduces processing time and improves response latency.

Important optimizations

1. Constant Folding: Computes fixed expressions during compilation rather than at runtime.
2. Dead Code Elimination: Removes unnecessary instructions and reduces code size.
3. Common Subexpression Elimination: Prevents repeated calculations during query analysis.
4. Control-Flow Optimization: Reduces unnecessary conditional jumps during intent classification.
5. Register Allocation: Keeps frequently accessed values in CPU registers for faster processing.
6. Procedure Call Optimization: Reduces the overhead of frequently called chatbot functions.
7. Code Size Optimization: Produces compact and efficient code, which improves resource utilization.

Impact on chatbot performance:

Compiler Optimization
↓
Efficient Intermediate Code
↓
Faster Query Processing
↓
Lower Response Latency
↓
Higher Concurrent User Capacity
↓
Better Customer Experience

For example, during peak periods, an optimized chatbot can process more customer requests with the same computing resources, reducing delays and improving scalability.

Conclusion:

Three Address Code provides an effective intermediate representation for chatbot query processing and response generation. Procedure calls make the chatbot modular, reusable, and scalable, while compiler optimization reduces execution time and resource consumption. Therefore, an optimized compiler is important for delivering fast, reliable, scalable, and responsive AI-based customer support services.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation